

Improving the performance of parallel programs (part 2)

Bibliography

- Culler et al – Chapter 3
- Grama et al – Chapter 3

Optimization goals

- Balance workload
- Reduce communication
- Reduce overhead introduced to increase parallelism, manage assignment, etc.

Load balancing issues

- Identifying enough concurrency in decomposition, and overcoming Amdahl's Law.
- Deciding how to manage the concurrency (statically or dynamically).
- Determining the granularity at which to exploit the concurrency.
- **Reducing serialization and synchronization cost.**

Decomposition techniques

- Recursive
 - Data
 - Exploratory
 - Speculative
 - Hybrid
-
- Identify enough concurrency
 - If too much, restrict by determining granularity of tasks

Management of concurrency

- Static assignment
 - Simple, predetermined assignment
 - Not much management overhead
 - Cost and amount of work should be predictable
- Semi-static assignment
 - Cost of work is predictable for near-term future
 - Periodically re-compute assignment
- Dynamic assignment
 - Collection of work queues
 - Contention
 - Management overhead
- **Task granularity influences contention and communication overhead**

Reduce serialization

- Serialization = wait time due to synchronization or mutual exclusion
- Synchronization
 - High wait time -> group (barriers, locks)
 - Lower wait time -> point to point (semaphores), group (read-write locks)
- Mutual exclusion
 - Separate locks for separate data items
 - Make critical sections smaller and less frequent
- Fine-grained synch and locks
 - Less wait time
 - Higher overhead (programming, execution time, not enough reuse...)

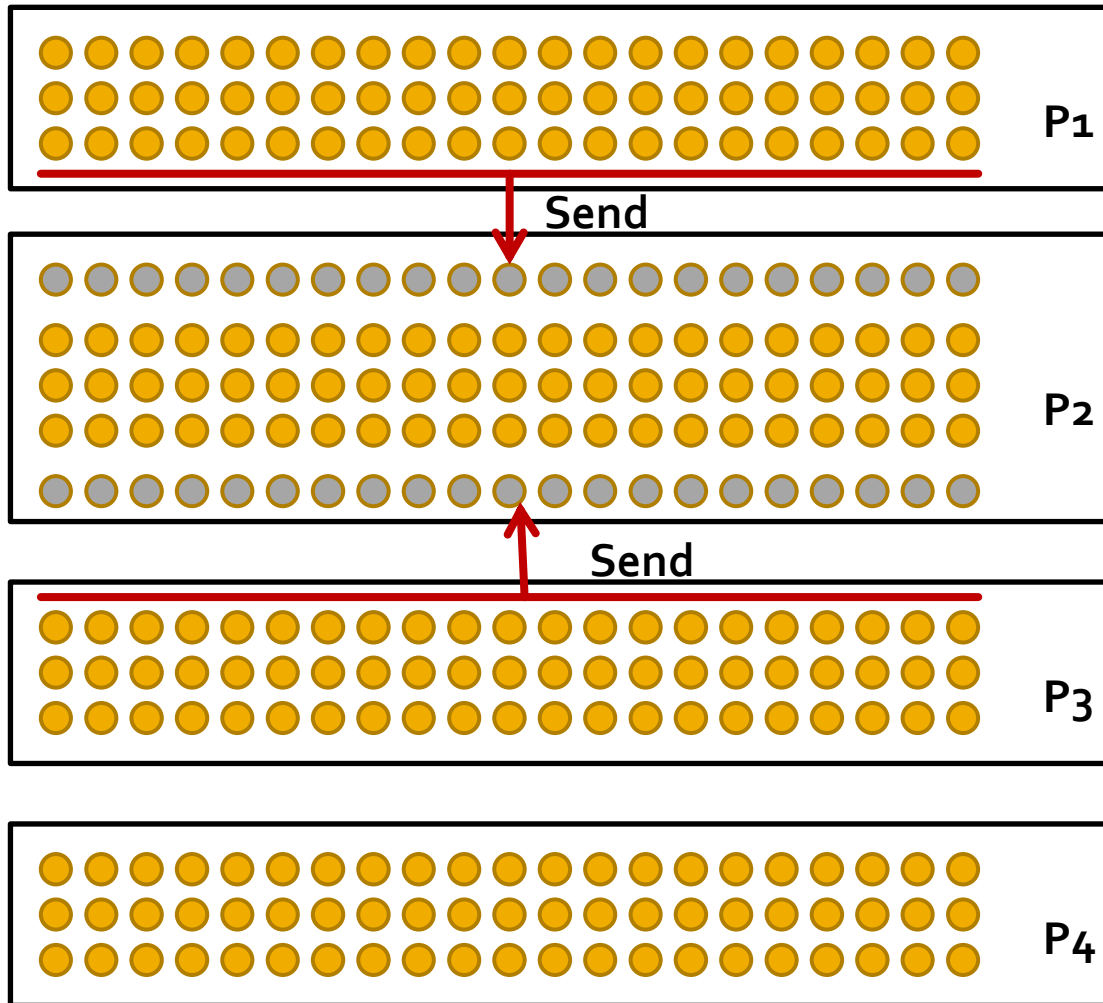
Second optimization goal

Reduce communication

Inherent communication

- Decompose a problem in tasks => communication among tasks
- Inherent communication = communication that must occur in the parallel algorithm
- Challenge: reduce inherent communication while still preserving load balancing

Solver – message passing implementation



Communication to computation ratio

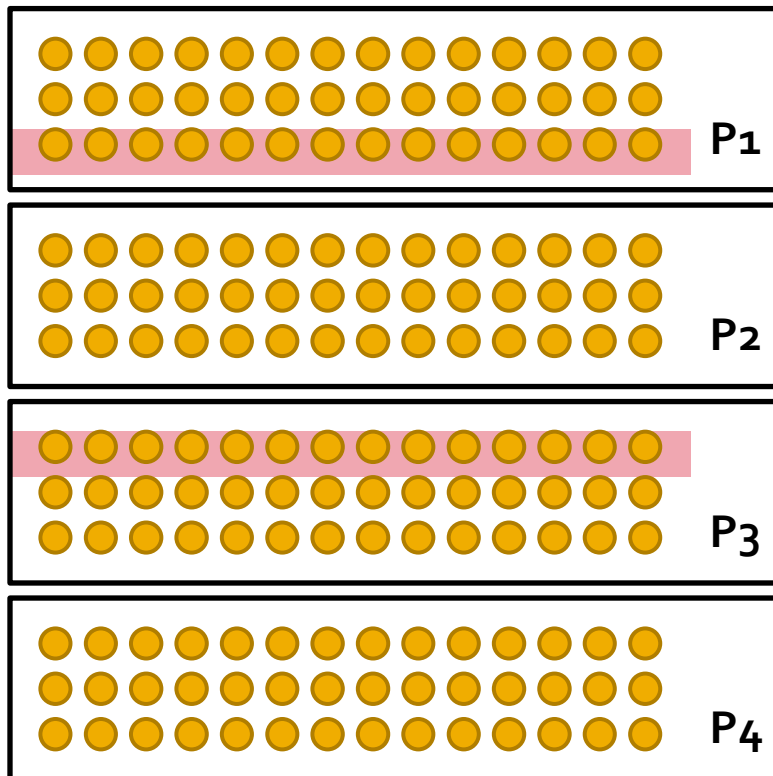
$$\frac{\text{Amount of communication (e.g. bytes)}}{\text{Amount of computation (e.g. instructions)}}$$

- Low communication to computation ratio is required to efficiently utilize modern parallel processors

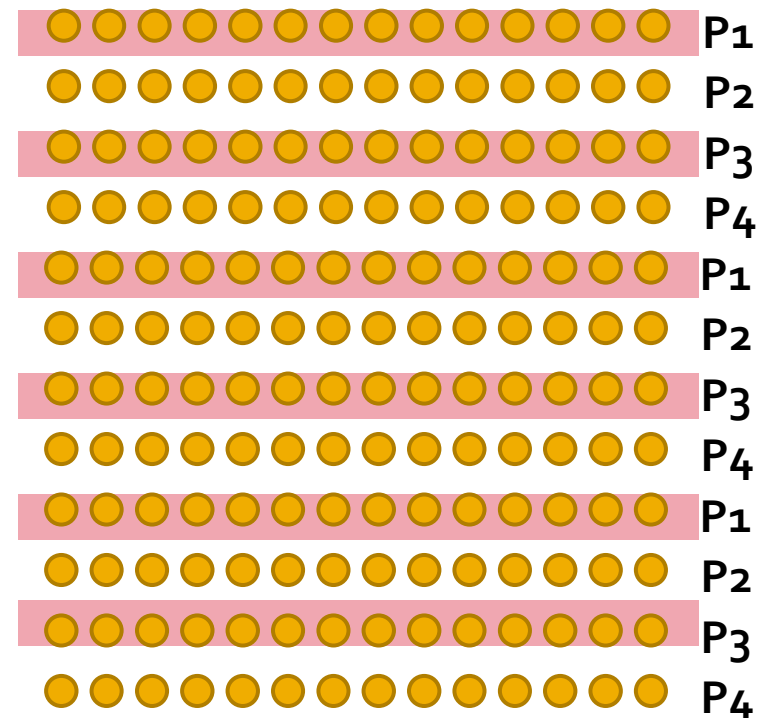
Reducing inherent communication

- Good assignment can reduce inherent communication

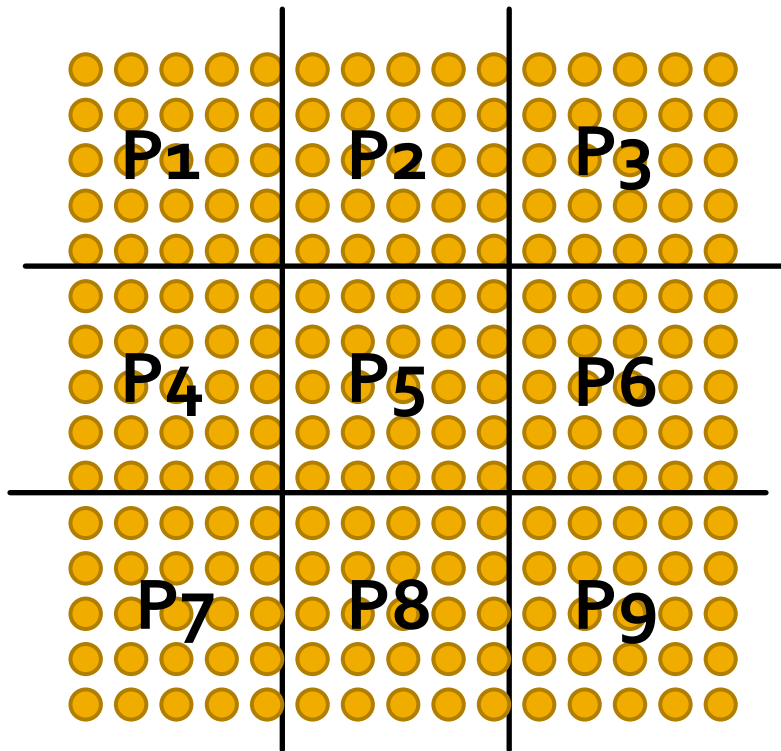
1D block assignment $\approx P/N$



1D interleaved assignment = 2



Reducing inherent communication



- Domain decomposition
- 2D block assignment
- Communication to computation ratio: $\frac{\sqrt{P}}{N}$

Extra work required for parallel execution

- Compute assignment
- Compute data values redundantly when communication cost is high
- Orchestration:
 - Process creation
 - Dynamic tasking
 - Synchronization operations
 - Structuring communication
 - Etc...

Speedup limit

$$\textit{Speedup}(P) \leq \frac{\textit{SequentialWork}}{\max(\textit{Work} + \textit{SyncWait} + \textit{Comm} + \textit{Extra})}$$

Parallel Random Access Memory (PRAM)

- This model assumes that data access is free
- Used for the analysis of parallel algorithms
- Can we oversee communication cost in real systems?

Artifactual communication

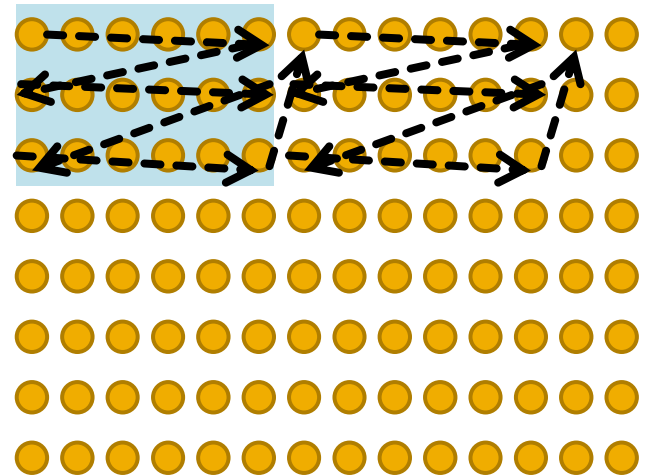
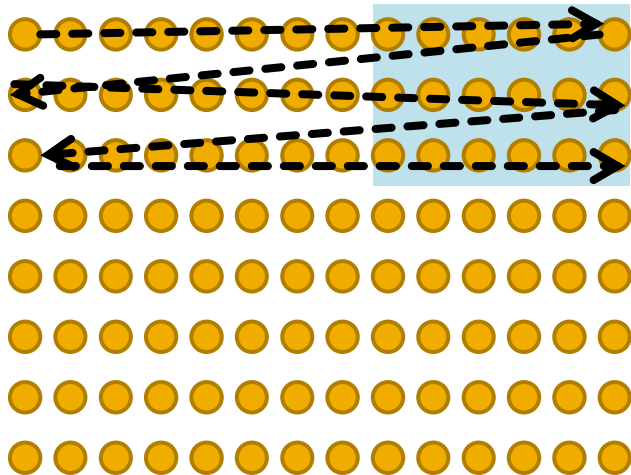
- Real parallel hardware - memory hierarchy, interconnection networks
- Artifactual communication – results from practical details of system implementation
- Sources:
 - Poor allocation of data: remote data access
 - Unnecessary data in a transfer: receiver does not use all transferred data
 - Unnecessary transfers due to system granularities: extra communication for cache coherence
 - Redundant communication of data
 - Finite replication capacity: cache is too small

Multiprocessor memory model

- Memory hierarchy of multiprocessors:
 - Cache
 - Local memory
 - Remote memory
- Memory wait time:
 - Cold start: first time data is accessed
 - Capacity miss: due to small cache size
 - Conflict miss: due to cache management policy
 - Communication miss: due to inherent or artifactual communication in parallel systems

Exploit temporal locality

- Access same memory location repeatedly in a short time-frame
- Use blocking to exploit temporal locality = accesses a subset of data that fits in a level of the hierarchy, use that data as much as possible

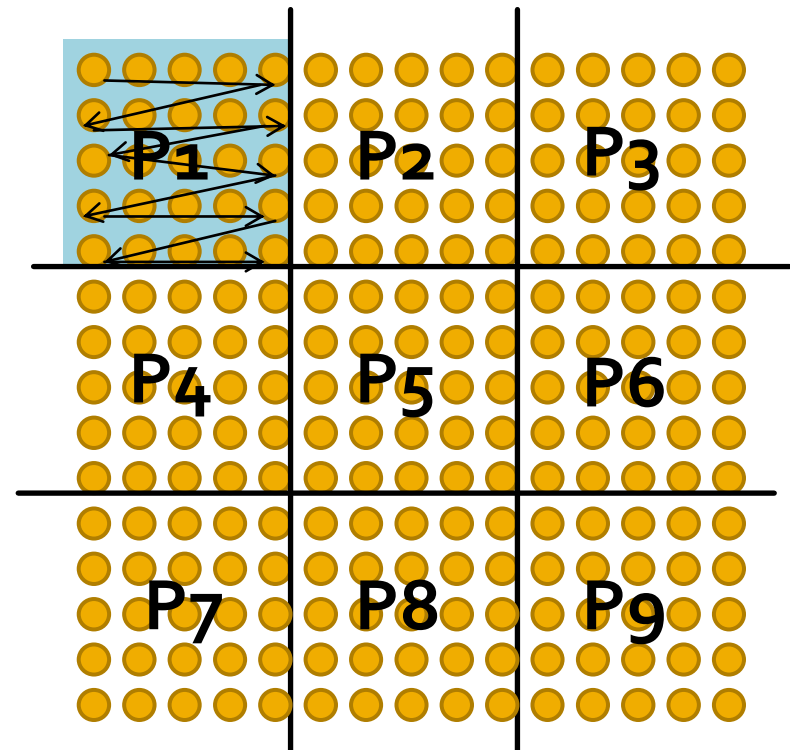
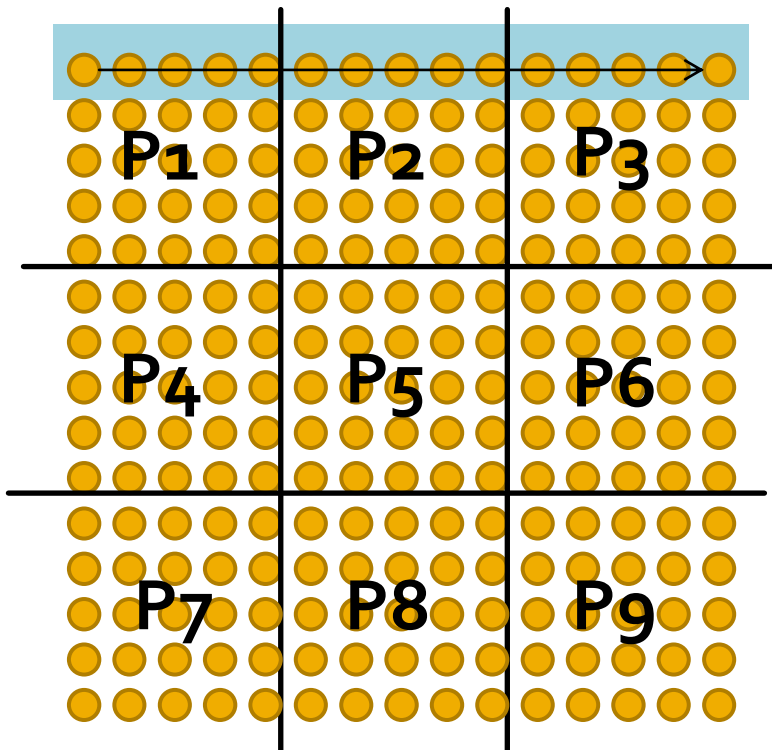


Exploit spatial locality

- Granularity of allocation: cache line size, page size
- Keep close together: the data accessed by a given processor (contiguous in the address space)
- Keep apart: unrelated data accessed by different processors
- Restructure data to interact better with the granularity of allocation

Exploit spatial locality

- Make sure: page and cache line within partition
- Change data structure from 2D to 4D to be able to fit data in the same cache line or page



Structuring communication to reduce cost

- How communication is organized and structured into messages influences:
 - Inherent communication
 - Artifactual communication
 - Communication to computation ratio
- Communication cost
 - +Overhead
 - +Delay
 - +Contention
 - -Amount of the communication cost that can be overlapped with computation

Reduce overhead

- Make messages fewer in number and hence larger
- Solutions:
 - Message passing: by programmer
 - Shared memory: by the system
- Solver example: send entire row in a single message

Reduce delay

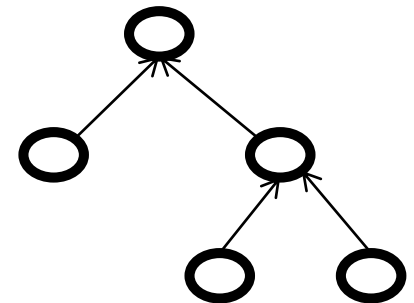
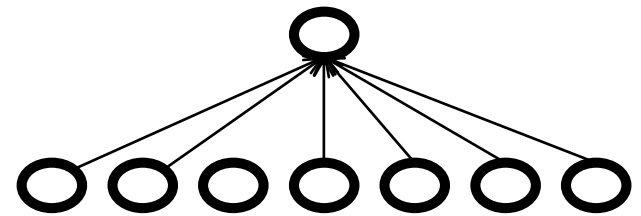
- $\text{Delay} = \text{nrHops} * \text{bitDelay}(\text{hop})$
- Solution: mapping the processes to processors so that locality in network topology is exploited
- Network topology not considered as important on modern machines because of the characteristics of the machines :
 - overhead dominates hop latency
 - number of nodes usually not very large
 - machines used as general-purpose, multiprogrammed servers->mapping solved by the OS transparently

Reduce contention

- Multiprocessors communication systems have many resources:
 - Network links
 - Controllers
 - Memory systems
 - Network interfaces
- Resource can do operations at a given throughput
- **Contention occurs when many requests to a resource are made within a small window of time (the resource is a “hot spot”)**

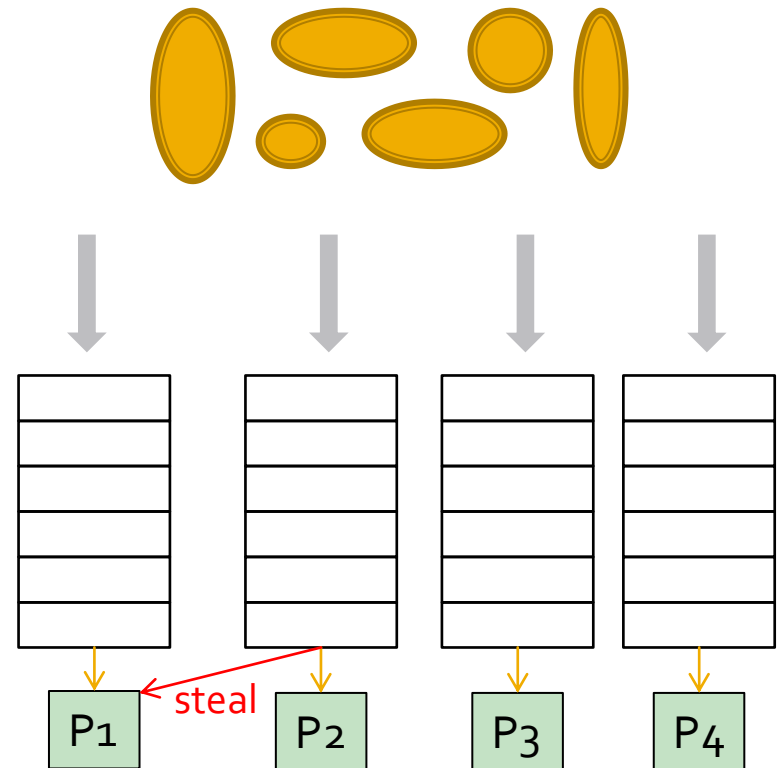
Reduce contention

- Recall: global sum example
- All processes trying to update a shared variable
- Flat communication:
 - Potential high contention
 - Low latency if no contention
- Tree structured communication:
 - Reduces contention
 - Higher latency under no contention



Example: distributed work queues

- Serve to reduce contention
- Take data from OWN work queue
- Put new work to OWN work queue



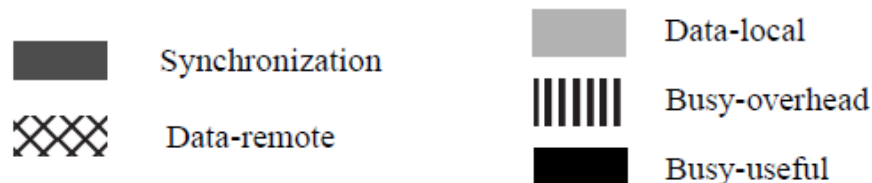
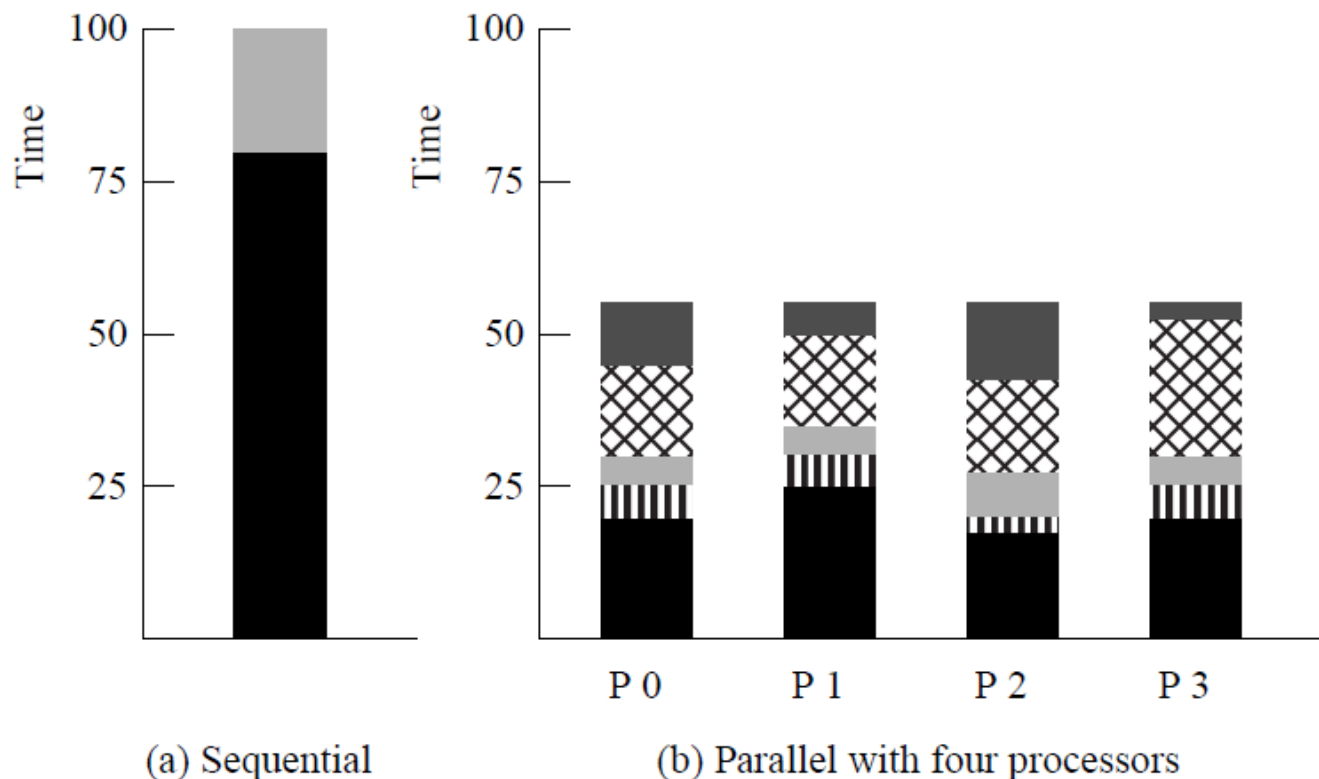
Overlapping communication with computation

- If processor would stall while communication occurs -> no parallel performance
- Hide the cost of communication
 - Overlap communication with computation
 - Overlap communication with communication
- Techniques:
 - HW: pipelining, multi-threading, pre-fetching, out-of-order, etc
 - Use asynchronous communication
 - Multithreading: requires additional concurrency in application (more concurrency than number of execution units)

Components of parallel execution time

- Busy-useful: execute instructions
- Busy-overhead: execute instructions needed only for the parallel algorithm
- Data-local: wait for local data
- Data-remote : wait for remote data
- Synchronization: wait for another process (includes load imbalance and serialization)

Components of parallel execution time



Parallel algorithm models

- Data-parallel
 - each task performs similar operations on different data
 - data parallelism
 - work may be done in phases (separated by interactions)
- Task graph
 - based on the task-dependency graph to reduce communication cost
 - divide-and-conquer decomposition
- Work pool
 - dynamic assignment
 - any task can be performed by any process
 - centralized/distributed queues

Parallel algorithm models

- Master-worker
 - master process generates work and assigns it to worker processes
- Producer-consumer (pipeline)
 - a stream of data is passed through a succession of processes
 - chains of processes (no cycles)
- Hybrid